

Matematyczne podstawy sieci neuronowych

Piotr Dyszewski

2026-06-08

Spis treści

Słowo wstępne	3
1 Wstęp	4
2 Uczenie nadzorowane	5
2.1 Regresja i klasyfikacja	8
3 Model neuronu i funkcja aktywacji	10
4 Płytkie uczenie	11
5 Stochastic Gradient Descent	12
6 Training	13
7 Głębokie sieci neuronowe	14
8 Initialization	15
9 Convolutional Neural Networks	16
10 Automatic Differentiation and Backpropagation	17
11 Adaptive Learning Rate Algorithms	18
12 Redukcja Wymiaru	19
13 Autoenkodery i encodery	20
14 Diffusion Models	21
15 Summary	22
References	23

Słowo wstępne

Notatki te powstają na potrzeby kursu o tym samym tytule, zaproponowanego w semestrze letnim roku akademickiego 2026/2027 w Instytucie Matematycznym Uniwersytetu Wrocławskiego.

Ich głównym celem jest zaznajomienie autora (a siłą rzeczy także czytelnika) z podstawowymi zasadami działania sieci neuronowych, które stanowią jeden z fundamentów dużych modeli językowych oraz modeli generatywnych. Nie chcemy traktować poszczególnych warstw sieci neuronowych jak czarnych skrzynek. Zamiast tego, poprzez analizę i implementację prostych, niskowymiarowych przykładów, będziemy starali się zrozumieć mechanizmy ich działania, a także ich mocne i słabe strony.

Lista źródeł, z których będę korzystał podczas przygotowywania tych notatek, ma charakter rozwojowy. Obecnie głównymi pozycjami są:

- *Mathematics of Neural Networks*, Bart Smets, <https://arxiv.org/pdf/2403.04807>
- *Alice's Adventures in a Differentiable Wonderland*, Simone Scardapane, <https://arxiv.org/pdf/2404.17625>
- *Neural Networks and Deep Learning*, Michael Nielsen, <http://neuralnetworksanddeeplearning.com/>

Notatki mają charakter roboczy i (jak wszystko we wstępnych fazach rozwoju) mogą zawierać błędy. Będę wdzięczny za zgłoszenie każdej usterki znalezionej w tekście.

Notatki są przygotowywane w Quarto, otwartym systemie do tworzenia publikacji naukowych i technicznych. Więcej informacji o Quarto można znaleźć na stronie <https://quarto.org/>.

1 Wstęp

W ciągu ostatnich kilkunastu lat uczenie maszynowe przeszło ogromny rozwój. Szczególną rolę odegrało w tym głębokie uczenie, oparte na sieciach neuronowych. Dzięki połączeniu prostych idei algorytmicznych, dużych zbiorów danych oraz rosnącej mocy obliczeniowej możliwe stało się rozwiązywanie zadań, które wcześniej były bardzo trudne, takich jak rozpoznawanie obrazów, przetwarzanie języka naturalnego, rozpoznawanie mowy czy analiza złożonych danych.

Sieci neuronowe są dziś jednym z podstawowych narzędzi sztucznej inteligencji. Wykorzystuje się je zarówno w widocznych zastosowaniach, takich jak duże modele językowe i generatory obrazów, jak i w systemach działających w tle: wyszukiwarkach, systemach rekomendacyjnych, analizie obrazów medycznych, projektowaniu leków czy technologiach autonomicznych.

Uczenie maszynowe polega na budowaniu modeli, które poprawiają swoje działanie na podstawie danych. Zamiast ręcznie projektować algorytm rozwiązujący dane zadanie, wybieramy ogólny model i uczymy go na przykładach. Gdy modelem tym jest sieć neuronowa z wieloma warstwami, mówimy o głębokim uczeniu.

Mimo imponujących sukcesów praktycznych nadal nie rozumiemy w pełni, dlaczego głębokie sieci neuronowe działają tak dobrze. Szczególnie interesujące jest to, że duże modele potrafią nie tylko zapamiętywać dane treningowe, ale także generalizować, czyli poprawnie działać na nowych przykładach.

Celem tych notatek jest wprowadzenie w podstawowe idee sieci neuronowych i głębokiego uczenia. Będziemy analizować zarówno intuicję, jak i formalny opis matematyczny modeli, aby zrozumieć, jak proste komponenty - warstwy, funkcje aktywacji, parametry, funkcje straty i algorytmy optymalizacji - pozwalają budować systemy zdolne do rozwiązywania złożonych problemów. Aby w pełni zrozumieć zasady działania sieci neuronowych skupimy się na niskowymiarowych przykładach.

2 Uczenie nadzorowane

Istnieją różne rodzaje uczenia maszynowego. My będziemy skupiać się na **uczeniu nadzorowanym**. Uczenie nadzorowane to paradygmat uczenia maszynowego, w którym dostępne dane składają się z par wejść oraz znanych, poprawnych wyjść. Nieznane jest natomiast to, jak dokładnie działa odwzorowanie między tymi wejściami i wyjściami. Celem jest wywnioskowanie, na podstawie dostępnych danych, ogólnej struktury tego odwzorowania, w nadziei, że będzie ono uogólniać się na niewidziane wcześniej sytuacje. Formalnie problem ten możemy sformułować następująco.

Problem uczenia nadzorowanego

Dana jest nieznana funkcja $f : X \rightarrow Y$ między przestrzeniami X oraz Y . Należy znaleźć dobre przybliżenie funkcji f , korzystając wyłącznie ze zbioru danych złożonego z N próbek:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N \quad \text{przy czym} \quad y_i = f(x_i) \quad \text{dla każdego } i = 1, \dots, N.$$

Przestrzeń X nazywana jest również **przestrzenią cech**, a jej elementy określa się mianem wektorów cech. Przestrzeń Y nazywana jest również **przestrzenią etykiet**, a jej elementy określa się mianem etykiet.

Przykład 1.1.

Niech X będzie przestrzenią wszystkich obrazów kotów i psów, a f klasyfikatorem, który odwzorowuje elementy tej przestrzeni w zbiór

$$Y = \{\text{"dog"}, \text{"cat"}\}.$$

Możemy to wyrazić numerycznie jako

$$X = [0, 1]^{3 \times H \times W},$$

czyli obraz o wysokości H , szerokości W oraz trzech kanałach koloru, oraz

$$Y = [0, 1]^2,$$

czyli jedna wartość prawdopodobieństwa dla każdej z dwóch klas.

Ogólne podejście do rozwiązywania tego problemu składa się z trzech głównych kroków.

1. Wybieramy model

$$F : X \times W \rightarrow Y$$

parametryzowany przez parametr z przestrzeni W . Litera W pochodzi od angielskiego słowa *weights*, ponieważ tak często określa się parametry w sieciach neuronowych.

2. Musimy określić ilościowo jakość wyjścia modelu, dlatego wybieramy **funkcję straty**

$$\ell : Y \times Y \rightarrow \mathbb{R},$$

gdzie $\ell(y_1, y_2)$ określa, „jak różne” są y_1 oraz y_2 .

3. Na podstawie zbioru danych oraz funkcji straty wybieramy $w \in W$ tak, aby

$$F(\cdot; w)$$

było możliwie najlepszym przybliżeniem funkcji docelowej f .

To wysokopoziomowe podejście pozostawia jednak kilka otwartych pytań.

- Jak wybrać model?
- Jak wybrać funkcję straty?
- Jak przeprowadzić optymalizację?

Na żadne z tych pytań nie ma kanonicznych odpowiedzi. W dużej mierze rozwiązuje się je metodą prób i błędów oraz za pomocą heurystyk, ponieważ musimy podjąć pewne decyzje, zanim będziemy mogli zrobić jakikolwiek postęp. Niezależnie od tego wybory te mają duży wpływ na końcowy wynik, mimo że nie są bezpośrednio uzasadnione przez dostępne dane ani przez jakieś podstawowe zasady. Zbiorczy zestaw założeń, które przyjmujemy, aby móc zająć się problemem, nazywa się w uczeniu maszynowym **biasem indukcyjnym** (*inductive bias*).

Na potrzeby tego kursu jako modele wybierzemy oczywiście sieci neuronowe; omówimy je dokładniej później. Drugą rzeczą, którą musimy zrobić, jest wybór funkcji straty. **Funkcja straty** to funkcja

$$\ell : Y \times Y \rightarrow \mathbb{R},$$

która działa podobnie do metryki, ale jest mniej restrykcyjna.

- Zwykle, choć nie zawsze, mamy

$$\ell : Y \times Y \rightarrow \mathbb{R}^+.$$

- Ponieważ chcemy minimalizować stratę, minimum powinno istnieć, tak aby wyrażenie

$$\min_{y \in Y} \ell(y_0, y)$$

było dobrze określone.

- Funkcja powinna być różniczkowalna, przynajmniej prawie wszędzie, ponieważ chcemy stosować spadek gradientowy.
- Własności metryczne, takie jak identyczność nierozróżnialnych, symetria czy nierówność trójkąta, nie muszą być spełnione.

Po wybraniu modelu i funkcji straty przechodzimy do optymalizacji, czyli pytania: jaki jest „najlepszy” wybór parametru $w \in W$? Najprostsza, choć niekoniecznie preferowana, odpowiedź brzmi: należy zminimalizować stratę na zbiorze danych, tzn. znaleźć

$$w^* = \arg \min_{w \in W} \sum_{i=1}^N \ell(F(x_i; w), y_i).$$

Przykład 1.2 (*liniowa metoda najmniejszych kwadratów*).

$$F(x; w) = \sum_{j=1}^n w_j \varphi_j(x)$$

oraz

$$\ell(y_1, y_2) = |y_1 - y_2|^2$$

dla pewnych funkcji bazowych $\{\varphi_j\}_j$. Otrzymujemy wtedy znane ustawienie liniowej metody najmniejszych kwadratów.

„Najlepszy” parametr $w \in W$ niekoniecznie jest tym, który minimalizuje stratę na zbiorze danych. W każdym problemie optymalizacyjnym częstym problemem jest **przeuczenie** (*overfitting*). Prowadzi nas to do **regularyzacji**.

Techniki regularyzacji dla sieci neuronowych omówimy szerzej później. Na razie wspomnimy tylko, że **regularyzacja parametrów** jest powszechną techniką znaną z regresji, często stosowaną również w sieciach neuronowych. Ten typ regularyzacji polega na dodaniu do straty na danych składnika karzącego, który zniechęca parametry do przyjmowania niepożądanych wartości, na przykład zbyt dużych. Zmodyfikowany problem optymalizacyjny ma postać

$$w^* = \arg \min_{w \in W} \sum_{i=1}^N \ell(F(x_i; w), y_i) + \lambda C(w),$$

gdzie $\lambda > 0$ oraz

$$C : W \rightarrow \mathbb{R}^+$$

jest funkcją karzącą w pewien sposób złożoność modelu.

Przykład 1.3 (*regularyzacja Tichonowa*).

Niech $W = \mathbb{R}^n$ oraz

$$C(w) = \|w\|^2.$$

W kontekście sieci neuronowych ten typ regularyzacji parametrów nazywany jest również **zanikiem wag** (*weight decay*).

2.1 Regresja i klasyfikacja

Uczenie nadzorowane powinno już brzmieć znajomo. Rzeczywiście, jeśli

$$X = \mathbb{R}^n \quad \text{oraz} \quad Y = \mathbb{R}^m,$$

to mamy do czynienia z **regresją**.

Regresja stanowi dużą część uczenia nadzorowanego, ale pojęcie uczenia nadzorowanego formuluje się ogólniej, tak aby obejmowało również inne zadania, takie jak **klasyfikacja**.

Klasyfikacja to w uczeniu maszynowym określenie na logistyczną regresję wieloklasową, w której

$$X = \mathbb{R}^n$$

i próbujemy przypisać każdy punkt $x \in X$ do jednej z m klas. Numeryczne wyjście dla tego typu problemu jest zwykle dyskretnym rozkładem prawdopodobieństwa na m klasach:

$$Y = \left\{ (y_1, \dots, y_m) \in [0, 1]^m \mid \sum_{i=1}^m y_i = 1 \right\}.$$

Przykład 1.1 jest właśnie tego typu.

W teorii uczenia statystycznego zakłada się, że próbki danych (x_i, y_i) są losowane niezależnie i z tego samego rozkładu, czyli i.i.d., z pewnego rozkładu prawdopodobieństwa μ na $X \times Y$. *Zastanów się, dlaczego jest to dość duże założenie.*

Interesuje nas wtedy minimalizacja **ryzyka populacyjnego**:

$$R(w) := \mathbb{E}_{(x,y) \sim \mu} [\ell(F(x; w), y)] := \int_{X \times Y} \ell(F(x; w), y) d\mu(x, y).$$

Celem byłoby znalezienie zestawu parametrów, który minimalizuje to ryzyko populacyjne:

$$w^* = \arg \min_{w \in W} R(w).$$

W rzeczywistości jednak nie znamy μ , więc nie możemy nawet obliczyć ryzyka populacyjnego, nie mówiąc już o jego minimalizacji. Robimy zatem najlepszą dostępną rzecz: minimalizujemy **ryzyko empiryczne**:

$$\widehat{R}(w) := \frac{1}{N} \sum_{i=1}^N \ell(F(x_i; w), y_i).$$

Zestaw parametrów minimalizujący ryzyko empiryczne nazywa się **minimalizatorem empirycznym**:

$$\widehat{w} = \arg \min_{w \in W} \widehat{R}(w),$$

co w kontekście uczenia nadzorowanego jest tym samym, co minimalizacja straty na zbiorze danych.

Gdy dodamy składnik regularyzacyjny, tak jak wcześniej, otrzymujemy tak zwaną **minimalizację ryzyka strukturalnego**:

$$\widehat{w} = \arg \min_{w \in W} \widehat{R}(w) + \lambda C(w).$$

Teoria uczenia statystycznego zajmuje się następnie badaniem takich zagadnień jak oszacowania różnicy

$$\widehat{R}(\widehat{w}) - R(w^*).$$

3 Model neuronu i funkcja aktywacji

4 Płytkie uczenie

5 Stochastic Gradient Descent

6 Training

7 Głębokie sieci neuronowe

8 Initialization

9 Convolutional Neural Networks

10 Automatic Differentiation and Backpropagation

11 Adaptive Learning Rate Algorithms

12 Redukcja Wymiaru

PCA Krótko: po co redukujemy wymiar? SNE / t-SNE jako metoda wizualizacji UMAP jako metoda wizualizacji

13 Autoenkodery i encodery

14 Diffusion Models

15 Summary

In summary, this book has no content whatsoever.

References